Lesson 8 — Homework

Data Cleaning və Transformation

Part 1 — Reminders

Bu dərsdə Pandas istifadə edərək dataset-dəki problemləri tapmağı, məlumatları təmizləməyi və yeni formaya çevirməyi öyrəndik.

Aşağıdakı reminder-lərdə hər anlayışın qısa izahı və Python kod nümunəsi verilib.

Example Dataset

```
import pandas as pd

df = pd.DataFrame({
    "customer_id": [1, 2, 2, 3, 4, 5],
    "age": ["24", "unknown", "unknown", None, "120", "38"],
    "city": [" Baku ", "GANJA", "GANJA", "baku", "Sumqayit", " baku "],
    "plan": ["Basic", "Premium", "Premium", " ", "basic", "Premium"],
    "monthly_spend": ["120", "450", "450", "unknown", "9000", "320"],
    "signup_date": [
        "2025-01-05",
        "2025-02-10",
        "2025-02-10",
        "not-a-date",
        "2025-04-03",
        "2025-04-12"
    ]
})

print(df)
```

Bu dataset-də duplicate, missing value, incorrect data type, string formatı və outlier problemləri var.

---

1. Dataset Inspection

Data Cleaning-ə başlamazdan əvvəl dataset-in strukturunu araşdırmaq lazımdır.

```
print(df.head())

print(df.shape)

df.info()

print(df.dtypes)
```

İzah:

- `head()` ilk 5 row-u göstərir.
- `shape` dataset-in ölçüsünü qaytarır: `(rows, columns)`.
- `info()` column-lar, non-null dəyərlər və data type-lar haqqında məlumat verir.
- `dtypes` hər column-un data type-ını göstərir.

Bizim nümunədə `df.shape` nəticəsi `(6, 7)` olacaq.

Column-dakı fərqli dəyərləri də yoxlaya bilərik:

```
print(df["city"].unique())

print(df["plan"].value_counts(dropna=False))
```

- `unique()` təkrarlanmayan fərqli dəyərləri qaytarır.
- `value_counts()` hər dəyərin neçə dəfə olduğunu göstərir.
- `dropna=False` missing value-ların da sayılmasını təmin edir.

Yadda saxlayın: Dataset-i araşdırmadan məlumatları silməyə və ya dəyişməyə başlamayın.

---

2. Missing Values

Missing value müəyyən bir məlumatın olmamasıdır.

Pandas belə dəyərləri `NaN`, `pd.NA` və ya tarixlərdə `NaT` ilə göstərə bilər.

Missing value-ları tapmaq

```
print(df.isna())

print(df.isna().sum())
```

`isna()` hər cell üçün `True` və ya `False` qaytarır.

`.sum()` əlavə etdikdə hər column-dakı missing value-ların sayını görürük.

Ümumi missing value sayını tapmaq:

```
print(df.isna().sum().sum())
```

Missing value-ları doldurmaq

Məsələn, numeric column-dakı missing value-ları sabit rəqəmlə doldura bilərik:

```
example = pd.DataFrame({
    "age": [20, None, 30, 40]
})

example["age"] = example["age"].fillna(25)

print(example)
```

Nəticə: Missing yaş 25 ilə əvəz olunur.

Amma real dataset-də hansı dəyərdən istifadə edəcəyimizə düşünərək qərar verməliyik.

Missing row-ları silmək

```
example = pd.DataFrame({
    "age": [20, None, 30, 40]
})

example = example.dropna(subset=["age"])

print(example)
```

Nəticə: Yaşı missing olan row silinir və 3 row qalır.

Diqqət: `unknown` string-i avtomatik missing value deyil. Onu ayrıca müəyyən edib çevirmək lazım ola bilər.

---

3. Duplicate Rows

Duplicate eyni məlumatın dataset-də təkrar olmasıdır.

Duplicate-ləri tapmaq

```
print(df.duplicated())

print(df.duplicated().sum())
```

`duplicated()` təkrar row-ları `True` olaraq göstərir.

Duplicate-ləri silmək

```
df = df.drop_duplicates().copy()

print(df.shape)
```

Nəticə: Nümunə dataset-də row sayı 6-dan 5-ə düşür.

Bəzən bütün row eyni olmasa da, ID təkrarlana bilər.

```
print(df["customer_id"].duplicated().sum())
```

Bu kod təkrarlanan customer ID-lərin sayını göstərir.

Yadda saxlayın: Tam duplicate row-u silmək ilə eyni ID-yə sahib fərqli məlumatlardan birini silmək eyni qərar deyil.

---

4. String Cleaning

String Cleaning mətn məlumatlarını standart formaya gətirmək üçün istifadə olunur.

Məsələn, `Baku`, `baku`, `BAKU` eyni şəhəri ifadə edir, amma proqram onları fərqli string-lər kimi görə bilər.

str.strip()

String-in əvvəlindəki və sonundakı boşluqları silir.

```
df["city"] = df["city"].str.strip()

print(df["city"].unique())
```

str.lower()

Bütün hərfləri kiçik hərflərə çevirir.

```
df["city"] = df["city"].str.lower()

print(df["city"].unique())
```

Nəticə: `baku`, `ganja` və `sumqayit` qalır.

str.upper()

Bütün hərfləri böyük hərflərə çevirir.

```
print(df["city"].str.upper())
```

Bu əməliyyat nəticəni göstərir, amma `df["city"]` column-unu dəyişmir, çünki nəticəni geri həmin column-a yazmamışıq.

Bir neçə method-u birləşdirmək

```
df["plan"] = df["plan"].str.strip().str.lower()

df["plan"] = df["plan"].replace("", pd.NA)

df["plan"] = df["plan"].fillna("unknown")

print(df["plan"].unique())
```

Burada əvvəl boşluqlar silinir, sonra hərflər kiçildilir.

Tamamilə boş qalan string-lər missing value kimi qeyd olunur və daha sonra `unknown` ilə doldurulur.

Yadda saxlayın: `.str.strip()` string-in ortasındakı bütün boşluqları və ya yazılış səhvlərini avtomatik düzəltmir.

---

5. Incorrect Data Types

CSV-dən gələn bəzi column-lar düzgün data type-da olmaya bilər.

Məsələn, `age` column-unda həm rəqəm, həm də `unknown` string-i ola bilər.

to_numeric()

```
df["age"] = pd.to_numeric(
    df["age"],
    errors="coerce"
)

df["monthly_spend"] = pd.to_numeric(
    df["monthly_spend"],
    errors="coerce"
)

print(df.dtypes)
```

İzah:

- `pd.to_numeric()` dəyərləri numerical type-a çevirməyə çalışır.
- `errors="coerce"` çevrilə bilməyən dəyərləri missing value-ya çevirir.

Məsələn, `"24"` rəqəmə çevrilir, `"unknown"` isə `NaN` olur.

Nəticəni yoxlayaq:

```
print(df[["age", "monthly_spend"]])

print(df[["age", "monthly_spend"]].isna().sum())
```

Yadda saxlayın: Numeric calculation etməzdən əvvəl column-un data type-ını yoxlamaq vacibdir.

---

6. Mean, Median və Missing Value Imputation

Missing numerical dəyərləri doldurmaq üçün mean və ya median istifadə edə bilərik.

Mean bütün rəqəmlərin ədədi ortasıdır.

Median sıralanmış rəqəmlərin ortasında olan dəyərdir.

```
numbers = pd.Series([20, 25, 30, 35, 1000])

print(numbers.mean())

print(numbers.median())
```

Nəticə:

- Mean: `222.0`
- Median: `30.0`

1000 dəyəri mean-i ciddi şəkildə yüksəltdi, amma median-a çox təsir etmədi.

Median ilə missing value doldurmaq

```
example = pd.DataFrame({
    "age": [20, None, 30, 40]
})

median_age = example["age"].median()

example["age"] = example["age"].fillna(median_age)

print(example)
```

Nəticə: Missing yaş median dəyər olan 30 ilə əvəz olunur.

Bu prosesə **imputation** deyilir.

Diqqət: Missing value-nu median ilə doldurmaq onun əsl dəyərini tapmaq demək deyil. Biz sadəcə mövcud məlumat əsasında təxmini dəyər seçirik.

---

7. Outliers

Outlier - datasetdəki digər dəyərlərdən ciddi şəkildə fərqlənən dəyərdir.

Məsələn, insanların yaşları əsasən 20–50 arasındadırsa, 120 yaş şübhəli görünə bilər.

Outlier tapmaq

```
print(df[df["age"] > 100])

print(df[df["monthly_spend"] > 2000])
```

Bu kod müəyyən threshold-dan(normadan) böyük dəyərləri göstərir.

Outlier dəyərini dəyişmək

Bizim nümunə dataset-imizdə 120 yaşın və 9000 aylıq xərcləmənin qəsdən yanlış olduğunu qəbul edək.

```
df.loc[df["age"] > 100, "age"] = pd.NA

df.loc[
    df["monthly_spend"] > 2000,
    "monthly_spend"
] = pd.NA
```

`.loc` vasitəsilə müəyyən şərtə uyğun row-un konkret column-unu dəyişirik.

İndi missing dəyərləri dolduraq:

```
df["age"] = df["age"].fillna(df["age"].median())

df["monthly_spend"] = df["monthly_spend"].fillna(
    df["monthly_spend"].median()
)

print(df[["age", "monthly_spend"]])
```

Yadda saxlayın: Hər böyük və ya kiçik dəyər yanlış deyil. Outlier-i dəyişməzdən əvvəl onun həqiqətən səhv olduğuna əsas olmalıdır.

---

8. Column Transformation və Mapping

Column Transformation mövcud məlumatdan yeni column yaratmaq və ya dəyərləri başqa formaya çevirməkdir.

Məsələn, `plan` column-undakı categorical dəyərləri rəqəmlərlə ifadə edə bilərik.

```
plan_mapping = {
    "basic": 0,
    "premium": 1,
    "unknown": -1
}

df["plan_code"] = df["plan"].map(plan_mapping)

print(df[["plan", "plan_code"]])
```

İzah:

`.map()` hər dəyər üçün dictionary-də uyğun qarşılığı axtarır.

Məsələn:

- `basic` → 0
- `premium` → 1
- `unknown` → -1

Diqqət: Bu rəqəmlər category-ləri ifadə edir. `premium=1` olması onun riyazi olaraq `basic=0`-dan bir vahid böyük olduğu demək deyil.

---

9. One-Hot Encoding

One-Hot Encoding categorical məlumatı ayrıca binary column-lara çevirir.

Məsələn, şəhərləri 0, 1, 2 kimi kodlaşdırmaq əvəzinə, hər şəhər üçün ayrıca 0/1 column yarada bilərik.

```
city_columns = pd.get_dummies(
    df["city"],
    prefix="city",
    dtype=int
)

print(city_columns)
```

Nəticədə yaranan column-lar:

- `city_baku`
- `city_ganja`
- `city_sumqayit`

Müştəri Bakıdandırsa, `city_baku` 1, digər şəhər column-ları 0 olur.

Yeni column-ları əsas DataFrame-ə əlavə etmək:

```
df = pd.concat([df, city_columns], axis=1)

print(df.head())
```

İzah:

`pd.concat()` DataFrame-ləri birləşdirir.

`axis=1` isə column-ları yan-yana əlavə edir.

---

10. Date/Time Transformation

Tarix məlumatları CSV-dən string formatında oxuna bilər.

Pandas-da onları datetime formatına çevirmək üçün `pd.to_datetime()` istifadə olunur.

```
df["signup_date"] = pd.to_datetime(
    df["signup_date"],
    format="%Y-%m-%d",
    errors="coerce"
)

print(df["signup_date"])

print(df["signup_date"].dtype)
```

İzah:

- `%Y` — dörd rəqəmli il.
- `%m` — ay.
- `%d` — gün.
- `errors="coerce"` — yanlış tarixləri `NaT` kimi saxlayır.

Missing tarixləri yoxlamaq

```
print(df["signup_date"].isna().sum())
```

Missing tarixli row-ları silmək

```
df = df.dropna(subset=["signup_date"]).copy()
```

Tarixdən yeni column-lar yaratmaq

```
df["signup_year"] = df["signup_date"].dt.year

df["signup_month"] = df["signup_date"].dt.month

df["signup_day"] = df["signup_date"].dt.day

print(
    df[["signup_date", "signup_year",
        "signup_month", "signup_day"]]
)
```

Nəticə: Tarixdən il, ay və gün məlumatları ayrıca çıxarılır.

Yadda saxlayın: Yanlış və ya missing tarixləri özümüzdən uydurduğumuz tarixlərlə əvəz etmək düzgün deyil.

---

11. Boolean Transformation və Yeni Column Yaratmaq

Mövcud column-a müəyyən şərt tətbiq edərək yeni binary column yarada bilərik.

Məsələn, aylıq xərcləməsi ən azı 300 olan müştəriləri müəyyən edək.

Comparison nəticəsi

```
print(df["monthly_spend"] >= 300)
```

Bu kod `True` və `False` qaytarır.

Boolean dəyərləri 1 və 0-a çevirmək

```
df["high_spender"] = (
    df["monthly_spend"] >= 300
).astype(int)

print(
    df[["monthly_spend", "high_spender"]]
)
```

İzah:

- Şərt doğrudursa → `True` → 1
- Şərt yanlışdırsa → `False` → 0

Neçə müştərinin high spender olduğunu görmək:

```
print(df["high_spender"].value_counts())
```

Bu üsul real layihələrdə qaydaya əsaslanan binary feature-lər yaratmaq üçün istifadə edilə bilər.

---

12. Data Quality Check və Export

Cleaning bitdikdən sonra nəticəni yoxlamaq vacibdir.

Missing values və duplicates

```
print("Rows:", len(df))

print("Duplicates:", df.duplicated().sum())

print("Duplicate IDs:", df["customer_id"].duplicated().sum())

print("Missing values:", df.isna().sum().sum())
```

Data types və category-lər

```
print(df.dtypes)

print(df["city"].unique())

print(df["plan"].unique())
```

CSV faylına yazmaq

```
df.to_csv("demo_customers_clean.csv", index=False)
```

`index=False` Pandas index-inin əlavə column kimi CSV-yə yazılmasının qarşısını alır.

Yadda saxlayın: Original CSV faylını ayrıca saxlamaq yaxşı vərdişdir. Beləliklə, cleaning zamanı səhv etdikdə raw(köhnə) dataya yenidən qayıtmaq mümkündür.

Part 2 — Homework Tasks

Dataset və başlanğıc

Yeni `lesson8_homework` folder-i yaradın.

Folder daxilində `customers_homework.csv` və `homework.py` faylları yaradın.

Aşağıdakı məlumatları CSV faylına yerləşdirin:

```
customer_id,name,age,city,plan,monthly_spend,signup_date
101,Amina,23, Baku ,basic,120,2025-01-15
102,Orkhan,34,baku,Premium,450,2025-02-02
103,Lala,,GANJA, premium ,310,2025-02-14
104,Nijad,28,Ganja,BASIC, ,2025-03-05
105,Sara,145,Sumqayit,Premium,220,2025-03-09
106,Ali,unknown,baku,basic,5000,2025-03-20
107,Nurlan,42, sumqayit ,PREMIUM,350,not-a-date
108,Samira,31,Baku,,unknown,2025-04-12
109,Kenan,25,ganja, basic ,280,
110,Aysu,37,BAKU,Premium,390,2025-05-01
102,Orkhan,34,baku,Premium,450,2025-02-02
111,Emil,29,Sumqayit,Basic,250,2025-05-16
```

Bütün task-ları eyni faylda ardıcıllıqla yerinə yetirin. Hər task əvvəlki task-da hazırladığınız DataFrame üzərində davam etməlidir.

Task 1 — Dataset Inspection

1. Pandas import edin.
2. CSV faylını `df` adlı DataFrame-ə oxuyun.
3. İlk 5 row-u göstərin.
4. Dataset-in shape-ini göstərin.
5. Bütün column-ların data type-larını yoxlayın.
6. Hər column-da neçə missing value olduğunu göstərin.

7. Neçə tam duplicate row olduğunu hesablayın.

Expected result: Dataset başlanğıcda 12 row və 7 column-dan ibarət olmalıdır. Bir tam duplicate row tapılmalıdır.

Task 2 — Remove Duplicates

1. Tam duplicate row-ları silin.
2. Nəticəni yenidən `df` variable-ında saxlayın.
3. Row sayını yenidən yoxlayın.
4. `customer_id` column-unda təkrarlanan ID olub-olmadığını yoxlayın.

Expected result: 11 row qalmalıdır və təkrarlanan customer ID olmamalıdır.

Task 3 — String Cleaning

`city` və `plan` column-larını təmizləyin.

1. Hər iki column-dakı artıq boşluqları silin.
2. Bütün dəyərləri lowercase formatına çevirin.
3. `plan` column-undakı boş string və missing dəyərləri `unknown` ilə əvəz edin.
4. Hər iki column-un unique dəyərlərini göstərin.

Expected result:

`city` yalnız `baku`, `ganja`, `sumqayit` dəyərlərindən ibarət olmalıdır.

`plan` yalnız `basic`, `premium`, `unknown` dəyərlərindən ibarət olmalıdır.

Task 4 — Numeric Cleaning və Outliers

1. `age` və `monthly_spend` column-larını numerical type-a çevirin.
2. Rəqəmə çevrilə bilməyən dəyərləri missing value kimi saxlayın.
3. Yaşı 100-dən böyük olan row-ları tapıb göstərin.
4. Aylıq xərcləməsi 2000-dən böyük olan row-ları tapıb göstərin.
5. Bu homework dataset-ində həmin iki limitdən böyük dəyərləri qəsdən yanlış qəbul edin və onları missing value(NaN) ilə əvəz edin.
6. Hər iki column-un median dəyərini hesablayın.

Expected result: 145 yaş və 5000 aylıq xərcləmə dəyərləri düzəliş üçün aşkar edilməlidir.

Task 5 — Handle Missing Values

1. `age` column-undakı missing value-ları həmin column-un median dəyəri ilə doldurun.
2. `monthly_spend` column-undakı missing value-ları həmin column-un median dəyəri ilə doldurun.
3. `age`, `monthly_spend` və `plan` column-larında missing value qalıb-qalmadığını yoxlayın.
4. Bu üç column-un nəticəsini göstərin.

Expected result: Həmin üç column-da missing value qalmamalıdır.

Task 6 — Date Cleaning

1. `signup_date` column-unu datetime formatına çevirin.
2. Tarix formatı kimi `YYYY-MM-DD` istifadə edin.
3. Yanlış və boş tarixlərin sayını göstərin.

4. Tarixi missing olan row-ları silin.
5. Tarixdən `signup_year` və `signup_month` adlı iki yeni column yaradın.

Expected result: İki row tarix problemi səbəbindən silinməli və 9 row qalmalıdır.

Task 7 — Encoding

1. `plan` column-u üçün aşağıdakı mapping-i tətbiq edin:
   - `basic` → 0
   - `premium` → 1
   - `unknown` → -1
2. Nəticəni `plan_code` adlı yeni column-da saxlayın.
3. `city` column-u üzərində One-Hot Encoding edin.
4. Nəticədə yaranan city column-larını əsas DataFrame-ə əlavə edin.
5. Yeni column-ları göstərin.

Expected result: `plan_code`, `city_baku`, `city_ganja`, `city_sumqayit` column-ları yaranmalıdır.

Combined Task — Customer Spending Classification

İndi mövcud təmizlənmiş dataset üzərində yeni feature yaradın.

Şirkət aylıq ən azı 300 manat xərcləyən müştəriləri ayrıca müəyyən etmək istəyir.

1. `high_spender` adlı yeni column yaradın.
2. `monthly_spend` 300 və ya daha çox olduqda 1, digər hallarda 0 yazılsın.
3. Nəticəni integer type-da saxlayın.
4. `customer_id`, `monthly_spend` və `high_spender` column-larını göstərin.
5. Neçə müştərinin high spender olduğunu hesablayın.

Expected result: 9 müştəridən 3-ü high spender olaraq işarələnməlidir.

Kiçik hint: Comparison əməliyyatı və `.astype(int)` istifadə edə bilərsiniz.